

■ SignLens Machine Learning Model Subsystem

Model Training, Feature Engineering & FastAPI Inference Server Guide

Welcome to the **SignLens Machine Learning Subsystem**. This directory contains the complete deep learning pipelines, MediaPipe landmark feature extraction scripts, model optimization tracks, and the production FastAPI inference microservice.

■ 1. Directory Structural Mapping

```
SignLens/model/
■■■■ model_endpoints/                # Production FastAPI Machine Learning Inference Server
■   ■■■■ app/
■   ■   ■■■■ api_v1/                # Endpoints & Auth (REST /predict & WebSocket /predict-stream)
■   ■   ■■■■ models/
■   ■   ■   ■■■■ model_manager.py    # Dynamic .h5 model scanner & thread-safe loader
■   ■   ■■■■ main.py                # FastAPI application initialization
■   ■■■■ saved_models/              # Pre-trained production TensorFlow model weights (.h5)
■   ■■■■ requirements.txt            # Python ML dependencies
■   ■■■■ test_local_inference.py     # Offline ML test script
■■■■ video_lstm_optimized/          # Flagship Dynamic Sequence Model (Stacked LSTM: (30, 258))
■   ■■■■ scripts/
■   ■   ■■■■ extract_keypoints_from_videos.py # MediaPipe batch extraction script
■   ■   ■■■■ model_video.py          # Keras model architecture definition
■   ■   ■■■■ train_model_video.py    # Model training script with callbacks
■   ■■■■ saved_models/              # Trained video LSTM .h5 checkpoints
■■■■ static_keypoints_optimized/     # Instant Static Character Model (Dense: (258,))
■   ■■■■ scripts/
■       ■■■■ extract_keypoints.py    # Static image landmark extractor
■       ■■■■ train_model_static.py   # Feed-forward neural network training
■■■■ model_image.py                 # Pixel-based CNN Baseline model (without keypoints)
■■■■ Research files/                # Academic rationale, model optimizations & research deep dives
```

■■■ 2. Prerequisite Setup (Step-by-Step for Evaluators)

Step 2.1: Python Environment Configuration

Ensure **Python 3.10** is installed on your host computer. Open Command Prompt / Terminal and navigate to `model/model_endpoints`:

```
# 1. Change directory to model_endpoints
cd model/model_endpoints

# 2. Create a clean virtual environment
python -m venv venv

# 3. Activate the virtual environment:
# Windows (Command Prompt):
venv\Scripts\activate
# Windows (PowerShell):
.\venv\Scripts\Activate.ps1
# macOS / Linux:
source venv/bin/activate

# 4. Upgrade pip
python -m pip install --upgrade pip
```

```
# 5. Install all required ML dependencies
pip install -r requirements.txt
```

■ 3. Running the Local FastAPI ML Server (Port 8000)

The ML server runs independently as an isolated private microservice.

Launch Command:

Make sure your virtual environment is active ((`venv`) appears in terminal), then run:

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Verification & Testing:

- Open your web browser to: `http://localhost:8000/docs`
- You will see the interactive **FastAPI Swagger Documentation** with endpoints:
- `POST /api/v1/predict?type=image`: Predicts static sign from uploaded image.
- `POST /api/v1/predict?type=video`: Predicts continuous sign sequence from video.
- `GET /api/v1/predict-stream`: WebSocket endpoint for real-time video frame streaming.

Automated Local Test Script:

In a separate terminal window (with `venv` active), run the automated offline test script:

```
python test_local_inference.py
```

This script passes sample keypoint tensors into the loaded TensorFlow models to verify prediction outputs and latency.

■■■■■ 4. Model Training & Feature Extraction Instructions

If you wish to re-train the models from scratch or train on custom gesture datasets, follow the instructions below for each modeling track:

Track 1: Dynamic Video LSTM Model ((30, 258) Matrix)

- **Model Topology:** Stacked LSTM (\$64 \rightarrow 128 \rightarrow 64\$) with 0.2\$ symmetric dropout, Dense(64, ReLU), and Softmax output.
- **Input Domain:** Sequences of 30 video frames containing 258 compressed MediaPipe landmarks (\$126\$ bilateral hand points + \$132\$ pose points).

```
# Navigate to video_lstm_optimized scripts directory
cd model/video_lstm_optimized/scripts

# 1. Extract MediaPipe landmark sequences from video dataset folder
python extract_keypoints_from_videos.py

# 2. Train the Stacked LSTM network (200 Epochs with ReduceLROnPlateau)
python train_model_video.py
```

Outputs: Generates `sign_language_model_video.h5` in the `saved_models/` folder.

Track 2: Static Keypoint Model ((258,) Vector)

- **Model Topology:** Dense Feed-Forward Network (Dense(128, ReLU) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Softmax).

- **Target:** Instant classification of static alphanumeric signs ($\text{\$A\text{\{--\}Z\$}$).

```
# Navigate to static_keypoints_optimized scripts directory
cd model/static_keypoints_optimized/scripts

# 1. Extract static landmark vectors from image dataset
python extract_keypoints.py

# 2. Train the static keypoint classifier
python train_model_static.py
```

Outputs: Generates `sign_language_model_static.h5`.

Track 3: Pixel-Based CNN Baseline (Image-based without MediaPipe)

- **Model Topology:** Conv2D + MaxPool2D + Dense Layers.

```
# From model directory root
cd model
python train_model_image.py
```

■ 5. Common Issues & Troubleshooting (ML Subsystem)

■ Problem 1: `ModuleNotFoundError: No module named 'tensorflow' or 'mediapipe'`

- **Cause:** The command was executed outside the Python virtual environment.
- **Solution:** Activate the virtual environment (`venv\Scripts\activate` on Windows) before running `uvicorn` or training scripts.

■ Problem 2: `FileNotFoundError: No models found matching pattern: sign_language_model_*.h5`

- **Cause:** The pre-trained `.h5` model files are missing from `model/model_endpoints/saved_models`.
- **Solution:** Ensure `sign_language_model_static.h5` and `sign_language_model_video.h5` exist in `model/model_endpoints/saved_models/`. You can copy them from `video_lstm_optimized/saved_models/` or `static_keypoints_optimized/saved_models/`.

■ Problem 3: MediaPipe / OpenCV C++ Runtime Warning

- **Cause:** Missing C++ redistributable package on Windows.
- **Solution:** Download and install Microsoft Visual C++ Redistributable from [\[https://aka.ms/vs/17/release/vc_redist.x64.exe\]](https://aka.ms/vs/17/release/vc_redist.x64.exe)(https://aka.ms/vs/17/release/vc_redist.x64.exe). Restart the machine.